

Background Reading: AMBF

Asynchronous Multi-Body Framework for Surgical Robotics Simulation

Chuntung Zhuang · CIS II Spring 2026 · Group 5: Vision-Based Eye Snake Robot Localization

Paper Reference

Authors	Adnan Munawar, Gregory S. Fischer
Title	An Asynchronous Multi-Body Simulation Framework for Real-Time Dynamics, Haptics and Learning with Application to Surgical Robots
Venue	IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)

Why This Paper

The fundamental challenge in training vision-based localization models for surgical robots is the scarcity of labeled real data. In a typical computer vision benchmark, tens of thousands of annotated images are available. In our setting, acquiring even a few hundred labeled frames requires access to a physical micro-continuum surgical robot, an live porcine eye, a Leica surgical microscope, a ROS recording setup, and an expert annotator who can identify the gripper tip and disk locations frame by frame. Our entire real dataset consists of 217 usable annotated frames across six recording sessions. This number is simply too small to train deep learning models from scratch without severe overfitting and instability.

Synthetic data generated through **AMBF, Asynchronous Multi-Body Framework for Surgical Robotics Simulation**, addresses this in three ways: first, it provides scale: by programmatically sweeping robot configurations and rendering automatically annotated frames, we generate 3,600 labeled images in a single pipeline run at zero additional annotation cost; second, it provides coverage: the real dataset undersamples unusual or extended robot poses due to the difficulty of holding the robot in extreme configurations during manual recording sessions. Simulation samples uniformly across the full reachable workspace, ensuring that rare poses are represented in training. lastly, it provides visual diversity: by varying retinal background textures and illumination conditions across 12 configurations per pose, the synthetic data exposes models to a broader range of appearance conditions than any single real recording session can capture.

Hence, we want to generate synthetic data, and this paper is the direct foundation of our synthetic data generation pipeline. AMBF solves our simulation platform issue by providing an environment where robot pose, camera viewpoint, and rendered RGB images are all available simultaneously and automatically. Every labeled frame in our synthetic dataset is produced through this framework.

Purpose of the AMBF Paper

What Problem AMBF Solves (Sec III.A)

The AMBF paper addresses a core challenge in real-time surgical robotics simulation: running physics, haptic feedback, and external communication simultaneously without any one component blocking the others. In a standard sequential implementation, all input devices are read one by one, forces are computed and applied,

and only then is the simulated world stepped forward — meaning that as more devices are added, the update rate of the entire loop degrades. Unbounded time-steps in a non-real-time OS then lead to what the paper calls cyclic deterioration, where each successive step takes longer than the last. AMBF resolves this by isolating each component into its own thread: the dynamic update-loop runs in one thread, and each haptic device update-loop runs in its own independent thread, preventing any single component from throttling the rest. The following image is taken directly from the paper to show the design:

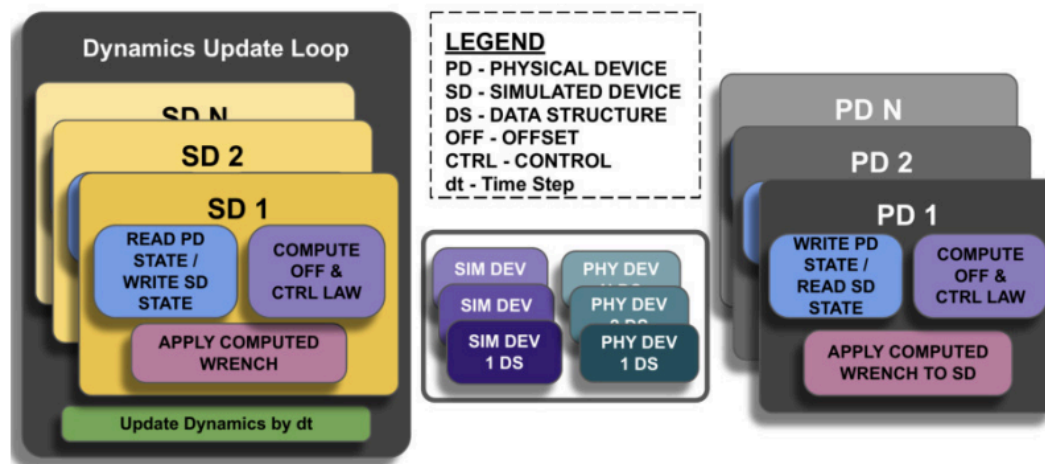


Fig. 3: A block diagram depicting the Design of Asynchronous Control Scheme, the Simulated end-effectors and Devices maintain independent and mutually exclusive Data Structures (DS) that are updated on successive writes and are capable of asynchronous reads

Design Philosophy and Architecture

The design philosophy of AMBF treats each simulated body as an independent object with its own kinematic and dynamic properties, in contrast to the conventional approach where all bodies are part of an interconnected graph requiring sequential updates. To realize this, all objects in the simulation are classified into exactly two types: afObject and afWorld, where 'af' stands for Asynchronous Framework.

An afObject represents any kinematic or dynamic rigid body in the scene — this includes robot links, end-effectors, obstacles, and any other physical entity — and can have any number of movable parts including zero. Each afObject operates independently, with its own state and control pipeline, allowing it to be commanded and read in parallel without interfering with other objects. An afWorld is the single global instance that manages the overall simulation environment, including all visual, kinematic, and dynamic objects, and supports features such as step-throttling and step-skipping for controlling simulation speed.

At their core, both afObject and afWorld expose two communication interfaces: afState for publishing the current state of the object, and afCommand for receiving control inputs. These two interfaces implement a generic input-output design that is easy to scale and communicate in parallel through ROS as an inter-process communication layer. ROS was selected specifically because it combines serialization and socket communication in a single middleware, making it scalable across operating systems and programming

languages. The framework is further accompanied by a Python client that allows learning agents built with tools like Keras, TensorFlow, and OpenAI GYM to interface directly with these `afObject` and `afWorld` handles, enabling real-time closed-loop training in the simulation.

What They Demonstrated

To validate the framework, the authors demonstrate AMBF on a bi-manual manipulation task in which two users simultaneously control four end-effectors — two via dVRK master telemanipulators with full haptic feedback, and two via Razer Hydra tracker devices — to grasp, lift, and place a multi-link puzzle using purely dynamic grasping with no simplification techniques. The system is further stress-tested by adding 200 simultaneous dynamic objects to the simulator, under which the dynamic loop drops to approximately 300 Hz, yet individual object communication speeds remain at approximately 2 kHz. The paper concludes by noting that the scope of the framework is intentionally generic, designed to support various surgical and non-surgical applications.

Core Features Relevant to Our Work

1. Synchronized RGB-D Capture

AMBF supports virtual cameras with configurable intrinsics (focal length, principal point, distortion coefficients). A calibrated camera can be mounted at any position in the scene and will produce RGB images synchronized with the simulator's internal clock. For our work, we calibrated the virtual camera to match the physical Leica surgical microscope using standard checkerboard calibration, ensuring that simulated images share the same projection geometry as real captures.

2. Automatic Keypoint Annotation

Because AMBF exposes the ground-truth 3D world coordinates of any object in the scene, we can project keypoint positions directly onto the image plane using the standard camera projection equation:

$$(\mathbf{u}, \mathbf{v})^T = \mathbf{K} \cdot \mathbf{P} \cdot (\mathbf{T}^{\text{camera}})^{-1} \cdot \mathbf{p}^{\text{world}}$$

This means every rendered frame comes with free, pixel-accurate labels for all three keypoints (gripper tip, first disk, last disk), so no manual annotation required. This is what makes large-scale synthetic dataset generation practical.

3. ROS Control Interface

AMBF exposes a native ROS (Robot Operating System) interface, allowing the robot to be commanded using the same topic-based messaging system used on the real hardware. In our pipeline, we drive the robot programmatically through increasing pitch and yaw bending angles, efficiently sweeping 300 configurations across the reachable workspace without any per-step manual intervention.

4. ADF Scene Description Format

Scenes are described using the Asynchronous Description Format (ADF), a YAML-based specification that defines robot geometry, inertial parameters, joint structure, and texture mappings independently for each component. This modularity is critical for our work: we can swap retinal textures, change lighting configurations, or substitute a different robot model without modifying the rest of the environment.

How We Used AMBF in This Work

We built a modular intraocular simulation environment on top of AMBF comprising three independently configurable components:

- **Robot model** — the Eye Snake modeled as 11 disc segments with alternating revolute joints, cable-driven actuation, and a two-finger gripper
- **Eyeball model** — a 12 mm radius spherical eye with interchangeable retinal texture maps derived from real porcine intraocular images
- **Camera model** — a virtual Leica surgical microscope with calibrated intrinsics transferred from the physical instrument

Using this environment, we sampled 300 robot configurations and rendered each under 12 visual conditions (4 retinal textures $\times$ 3 lighting setups), yielding 3,600 automatically annotated frames. These were further expanded to 900 training samples via domain randomization (brightness reduction, Gaussian blur, scaling, noise injection). This synthetic dataset forms the pretraining stage for our sim-to-real finetuning pipeline.

Some examples are as followed, which are all what our simulation does and collect, which will be further elaborate in the checkpoint presentation

